Exact SAT Solving and Constraint Programming for the N-Queens Problem

Pham Vinh Tri[a]

[a]*Faculty of Information Technology, VNU University of Engineering and Technology, 144 Xuan Thuy, Cau Giay, Hanoi, 100000, Vietnam*

ABSTRACT

The N-Queens problem is a classic combinatorial optimization benchmark requiring the placement of N non-attacking queens on an $N \times N$ chessboard. This paper presents a comprehensive study of exact solving methodologies centered on Boolean Satisfiability (SAT). The primary contributions include the mathematical formulation, implementation, and empirical evaluation of five At-Most-One (AMO) encodings: Binomial (Pairwise), Sequential Counter, Binary, Commander, and Product. Furthermore, we conduct an extensive cross-paradigm benchmark against two other exact paradigms: Constraint Programming (Google OR-Tools CP-SAT, IBM CPLEX CP) and Integer Linear Programming (IBM CPLEX MIP, Gurobi). Computational experiments spanning baseline dimensions ($N = 8$ to 30), scalability extensions ($N = 40, 50$), and massive stress-test instances ($N = 100, 200$) reveal a severe trade-off between encoding size and solving latency. We quantitatively demonstrate that auxiliary variables—introduced to compress clause counts—can severely degrade Conflict-Driven Clause Learning (CDCL) heuristics. A cross-solver study across five modern backends (Glucose3, Glucose4, CaDiCaL, MapleSAT, and CryptoMiniSAT5) demonstrates that solver rankings reverse depending on the encoding, proving that solver–encoding synergy, rather than CNF size alone, dictates performance. Finally, a fairness evaluation under a first-feasible-solution stopping criterion confirms that the performance advantage of CP and MIP stems from high-level problem formulation and direct branching on original decision variables rather than termination discrepancies.

1 Introduction

Graph layout and resource allocation represent fundamental families of combinatorial optimization problems with pervasive applications in VLSI design, telecommunications architectures, and automated scheduling [1]. Within this domain, the N-Queens problem serves as a classical and rigorous benchmark for evaluating the efficiency of combinatorial search and constraint reasoning algorithms.

The objective of the problem is to place N queens on an $N \times N$ chessboard such that no two queens attack each other under standard chess rules (i.e., sharing neither row, column, nor diagonal). As the board size scales from small dimensions to large problem spaces, combinatorial explosion expands the state space factorially. Consequently, traditional depth-first backtracking algorithms rapidly succumb to computational exhaustion.

This study presents a complete and rigorous SAT-based methodology for solving the N-Queens problem exactly. By examining the interplay between Conflict-Driven Clause Learning (CDCL) heuristics and auxiliary boolean variables, we quantitatively analyze the performance paradox across diverse Conjunctive Normal Form (CNF) encodings. The remainder of this report is organized as follows: Section 2 reviews related work. Section 3 formalizes the mathematical problem formulation. Section 4 establishes five SAT encodings for At-Most-One (AMO) constraints along with symmetry breaking. Section 5 presents the Constraint Programming (CP) formulation. Section 6 provides comprehensive experimental benchmarks, scalability evaluations, cross-solver comparisons, fairness analyses, and runtime bottleneck discussions. Section 7 concludes the paper.

ORCID(s):

2 Related Work

Transforming cardinality constraints into Conjunctive Normal Form (CNF) is a foundational and actively developed area in propositional satisfiability. Warners [2] proposed linear-time transformations for linear inequalities using auxiliary variables. Sinz [3] subsequently introduced the Sequential Counter encoding, achieving linear $O(n)$ clause complexity and establishing a standard building block in modern SAT applications. Gent and Nightingale [4] analyzed the trade-off between encoding compactness and unit propagation strength. For the specific At-Most-One (AMO) constraint, Klieber and Kwon [5] proposed the Commander encoding, while Chen [6] designed the Product encoding based on 2D coordinate decomposition.

In terms of solver architectures, Conflict-Driven Clause Learning (CDCL) equipped with Variable State Independent Decaying Sum (VSIDS) decision heuristics [7] constitutes the core engine of Glucose [8], the primary SAT solver evaluated in this study. In parallel, Constraint Programming (CP) [9] preserves high-level integer constraint structures, filtering infeasible domains via maximum bipartite matching on `AllDifferent` constraints [10]. Furthermore, Integer Linear Programming (ILP) / Mixed Integer Programming (MIP) [11] formulates combinatorial constraints as systems of linear inequalities solved via branch-and-cut; by branching directly on primary decision variables without auxiliary boolean variables, MIP avoids heuristic disorientation. Our study evaluates all three paradigms (SAT, CP, ILP) side by side on a uniform benchmark, illuminating how encoding structure and variable introduction impact solver performance.

3 Problem Formulation

The N-Queens problem can be formally modeled as a Constraint Satisfaction Problem (CSP). Given an $N \times N$ chessboard, let $x_{i,j} \in \{0, 1\}$ denote binary decision variables indicating whether a queen is placed at row i and column j, where $i, j \in \{1, \dots, N\}$. The constraint system is defined by:

$$\sum_{j=1}^{N} x_{i,j} = 1 \quad \forall i \in \{1, \dots, N\} \tag{1}$$

$$\sum_{i=1}^{N} x_{i,j} \leq 1 \quad \forall j \in \{1, \dots, N\} \tag{2}$$

$$\sum_{i-j=k} x_{i,j} \leq 1 \quad \forall k \in \{-(N-1), \dots, N-1\} \tag{3}$$

$$\sum_{i+j=k} x_{i,j} \leq 1 \quad \forall k \in \{2, \dots, 2N\} \tag{4}$$

Equation (1) enforces the Exactly-One constraint for every row. Equations (2), (3), and (4) enforce At-Most-One (AMO) constraints across columns and both sets of diagonals. Because an Exactly-One constraint naturally decomposes into an At-Least-One (ALO) clause and an AMO constraint, the core mathematical challenge lies in optimizing the CNF representation of AMO constraints.

4 SAT Encodings for At-Most-One Constraints

To leverage modern CDCL SAT solvers, the constraint system must be transformed into Conjunctive Normal Form (CNF). Given a set of boolean variables $X = \{x_1, \dots, x_n\}$, the AMO constraint enforces $\sum_{i=1}^{n} x_i \leq 1$. Below is the mathematical formulation of the five evaluated encodings.

4.1 Binomial (Pairwise) Encoding

The most intuitive formulation enforces mutual exclusion across all pairs of variables. For every pair $i \neq j$, both variables cannot simultaneously evaluate to True:

$$AMO_{\text{Binomial}}(X) = \bigwedge_{1 \leq i < j \leq n} (\neg x_i \vee \neg x_j) \tag{5}$$

This direct encoding introduces zero auxiliary variables. However, it incurs quadratic space complexity with $\binom{n}{2} = O(n^2)$ binary clauses.

4.2 Sequential Counter Encoding

To reduce clause volume, Sinz [3] introduced $n-1$ auxiliary variables $s_1, \dots, s_{n-1}$ acting as an accumulator register. Variable s_i evaluates to True if at least one x_k $(k \leq i)$ is True:

$$(\neg x_1 \vee s_1) \tag{6}$$

$$\bigwedge_{i=2}^{n-1} ((\neg x_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg x_i \vee \neg s_{i-1})) \tag{7}$$

$$(\neg x_n \vee \neg s_{n-1}) \tag{8}$$

This encoding achieves a linear clause count of $3n-4 = O(n)$, at the cost of introducing $n-1 = O(n)$ auxiliary variables.

4.3 Binary, Commander, and Product Encodings

Binary Encoding: Encodes the index of the active variable (if any) in binary using $\lceil \log_2 n \rceil$ auxiliary variables, producing $O(n \log n)$ clauses.

Commander Encoding: Partitions variable set X into disjoint subsets of size $\approx \sqrt{n}$ [5]. Each subset is governed by a commander variable, achieving a balanced trade-off with $O(n\sqrt{n})$ clauses and $O(\sqrt{n})$ auxiliary variables.

Product Encoding: Arranges the variables into a 2D matrix of dimensions $p \times q$ with $p \approx q \approx \sqrt{n}$ [6]. Setting x_k to True activates its corresponding row and column auxiliary variables, yielding $O(n)$ clauses with $O(\sqrt{n})$ auxiliary variables. Despite its compactness, this encoding *does not preserve* Generalized Arc Consistency (GAC) under unit propagation: the row and column auxiliaries are shared across many variables, so assigning one queen variable propagates only weakly, and infeasible partial assignments cannot be pruned until additional branching occurs [4].

> **Technical Note: PySAT Cardinality Encoding Semantics**
>
> **Note on PySAT encoding labels.** In addition to the five handwritten implementations in `encoder.py`, our codebase includes `pysat_encoder.py`, which interfaces directly with PySAT's cardinality algorithms (`pysat.card.CardEnc`). It should be noted that certain PySAT encoding identifiers do not map one-to-one to the algorithms named above: specifically, `CardEnc` types `cardnetwrk` (Cardinality Networks) and `mtotalizer` (Modulo Totalizer) are generalized sorting/counter networks with topological structures distinct from our handwritten Commander and Product encodings. Therefore, the PySAT module serves primarily as an independent verification baseline for logical correctness. All quantitative comparisons and structural evaluations in this report originate strictly from the five native implementations in `encoder.py`.

4.4 Symmetry Breaking

Solutions to the N-Queens problem are invariant under horizontal reflection: if a board configuration is valid, its reflection across the vertical axis is also valid. We eliminate half of the search space without loss of completeness by fixing the queen in the first row to the left half of the board:

$$\neg x_{0,c} \quad \forall c \in \{\lceil N/2 \rceil, \dots, N-1\} \tag{9}$$

These unit clauses are injected identically across all five SAT encodings, ensuring that all solvers navigate the exact same pruned search space and guaranteeing rigorous benchmarking fairness.

5 Constraint Programming (CP) Formulation

Unlike SAT solving, which decomposes problems into propositional clauses, Constraint Programming (CP) preserves high-level integer domain abstractions. Defining an array of integer variables $Q = [Q_0, \dots, Q_{N-1}]$, where $Q_i \in \{0, \dots, N-1\}$ denotes the column coordinate of the queen placed on row i, the entire problem is captured using three global constraints:

- `AddAllDifferent(`Q`)`
- `AddAllDifferent(`$Q_i - i \; \forall i$`)`
- `AddAllDifferent(`$Q_i + i \; \forall i$`)`

Table 1
Theoretical comparison of the five AMO encodings. "GAC" indicates whether the encoding preserves Generalized Arc Consistency under unit propagation.

Encoding	Aux. Variables	Clause Complexity	GAC
Binomial (Pairwise)	0	$O(n^2)$	✓
Sequential Counter	$O(n)$	$O(n)$	✓
Commander	$O(\sqrt{n})$	$O(n\sqrt{n})$	✓
Binary	$O(\log n)$	$O(n \log n)$	×
Product	$O(\sqrt{n})$	$O(n)$	×

Internally, CP engines propagate `AddAllDifferent` constraints using maximum bipartite matching filtering algorithms [10]. This achieves generalized arc consistency (GAC), pruning infeasible subtrees without materializing millions of propositional clauses.

6 Computational Experiments and Bottleneck Analysis

6.1 Theoretical Summary of the Five Encodings

Before turning to the empirical results, Table 1 consolidates the structural profile of each encoding: its auxiliary-variable count, clause complexity, and—most importantly for the discussion that follows—whether the encoding preserves *Generalized Arc Consistency* (GAC) under unit propagation. GAC is the property that, for the underlying AMO constraint, every value removed by the propagation of the CNF is logically infeasible; an encoding that is GAC-enforcing allows a solver to prune inconsistent partial assignments as early as possible [4]. As the table indicates, only Binomial, Sequential, and Commander maintain GAC under unit propagation, whereas Binary and Product—despite their compact clause counts—fail to do so.

The comparison in Table 1 frames the central puzzle of this study. The two encodings with the smallest auxiliary-variable footprint and the most compact clause counts—Binary and Product—are precisely the two that forfeit GAC. This theoretical deficit foreshadows the empirical catastrophe documented in Section 6.4: without GAC, propagation cannot detect infeasibility early, and the solver is forced to enumerate deep into incorrect branches.

6.2 Experimental Setup

Experiments were conducted across baseline dimensions $N \in \{8, 10, 15, 20, 25, 30\}$, and subsequently extended to two large scales ($N = 40, 50$) and two massive stress-test scales ($N = 100, 200$) to evaluate scalability frontiers and pinpoint breakdown points. Three distinct families of exact solvers were evaluated: (i) SAT solving with 5 encodings utilizing the Glucose3 CDCL backend; (ii) Constraint Programming with CP-SAT (Google OR-Tools) and CPLEX CP Optimizer (IBM); and (iii) Integer Linear Programming with CPLEX MIP (IBM) and Gurobi. For instances $N \leq 30$, each configuration was executed across 3 repetitions and averaged; for extended benchmarks, single runs were used for timeouts while converging runs averaged 3 repetitions. To quantify measurement noise, standard deviations as well as minimum and maximum solve times are reported. High-precision monotonic timing via `time.perf_counter()` was employed to eliminate discretization artifacts. All solvers were restricted to single-threaded execution to ensure fairness. Solutions across all paradigms—not merely SAT—were independently validated by a shared verification oracle. Model generation time (Python) and solving time (solver core) were measured independently. A cross-solver evaluation across five backends (Glucose3, Glucose4, CaDiCaL, MapleSAT, CryptoMiniSAT5) and a first-feasible-solution fairness benchmark for CP/MIP (Section 6.5) were also conducted.

Table 2 exposes the **true breakdown point of SAT encodings**. At $N = 100$, Binomial generates **1,646,850 clauses**—over 38 times more than at $N = 30$—yet solves in merely 0.183**s**. In contrast, *all* auxiliary-variable encodings (Sequential, Commander, Binary, Product) **exceed the** 40**s feasibility threshold**, despite producing only $\frac{1}{14}$ to $\frac{1}{6}$ as many clauses. This conclusively proves: **CNF size does not dictate solving speed; the introduction of auxiliary variables destroys search heuristics**. At $N = 200$, Binomial itself breaks down: taking 4.6s merely to *generate* 13,253,700 clauses and failing

Table 2
Scalability stress-test on massive instances $N = 100$ and $N = 200$ (in seconds; average of 3 runs for converging methods, 1 run for timed-out methods). Time limit: 40s for SAT, 60s for CP-SAT. "–" denotes timeout or ungenerated clauses.

Encoding / Solver	N	Clauses	Gen (s)	Solve (s)
Binomial	100	1 646 850	0.575	0.183
CP-SAT	100	–	0.001	0.641
Sequential	100	117 762	0.044	– (timeout)
Commander	100	224 924	0.066	– (timeout)
Binary	100	268 974	0.079	– (timeout)
Product	100	116 570	0.048	– (timeout)
Binomial	200	13 253 700	4.618	– (timeout)
CP-SAT	200	–	0.002	5.897

to solve within 40s—delineating the structural ceiling of pure SAT reductions. In contrast, CP-SAT (which introduces no auxiliary Booleans) solves $N = 100$ in 0.64s and $N = 200$ in 5.9s, sustaining robust scalability.

6.3 Encoding-Size vs Runtime Analysis

Experimental findings in Figure 1 corroborate theoretical predictions: as N scales from 8 to 50, Binomial clause volume grows quadratically, yielding a massive CNF with 203,425 clauses at $N = 50$ (compared to 43,255 at $N = 30$). In contrast, Sequential, Binary, Commander, and Product maintain compact formulas ranging between 28,887 and 57,219 clauses at $N = 50$, achieved through auxiliary variable compression. Consequently, the gap in formula size widens substantially with larger board sizes.

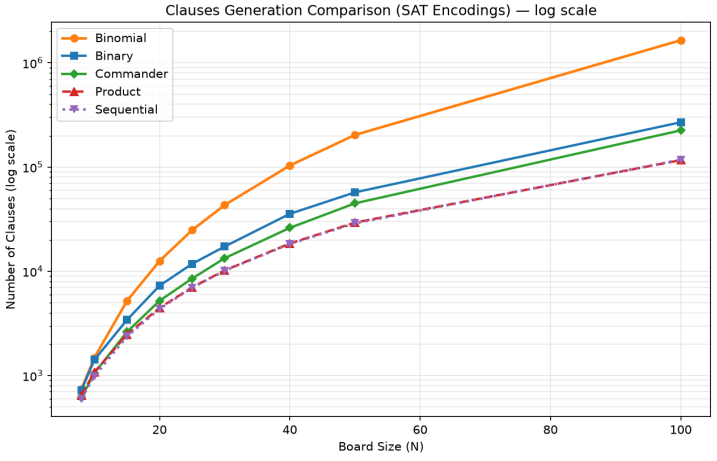


Figure 1: Growth of clause count across SAT encodings as board size N increases (logarithmic scale).

Table 3 reports runtime metrics at $N = 30$. Timing is decomposed into Python clause generation (*Gen*) and solver execution (*Solve*).

Table 3 highlights a crucial insight: for Binomial, the majority of runtime (0.016s out of 0.029s, or approximately 56%) is consumed by Python clause generation rather than the SAT solver core. A similar pattern emerges in CPLEX-MIP, where modeling overhead (0.017s) exceeds solving time (0.011s). This underscores the necessity of decoupling *formulation overhead* from *solving duration* when evaluating constraint encodings.

6.4 Runtime Bottleneck Analysis

Inspection of Figure 2 uncovers a severe performance paradox in SAT solving: a smaller clause footprint does not guarantee faster execution.

Table 3
Runtime performance at $N = 30$ (average of 3 repetitions), in seconds.

Encoding / Solver	Clauses	Variables	Gen (s)	Solve (s)
Binomial	43 255	900	0.016 11	0.012 56
Sequential	10 137	4322	0.001 31	0.207 53
Binary	17 301	1666	0.009 27	9.650 24
Commander	13 373	1606	0.005 43	0.248 49
Product	10 215	2426	0.002 42	14.448 40
CP-SAT	–	–	0.001 00	0.077 55
CPLEX-CP	–	–	0.001 20	0.049 42
CPLEX-MIP	–	–	0.016 53	0.011 33
Gurobi-MIP	–	–	0.007 50	0.000 63

At $N = 10$, solve times across all methods are negligible; however, divergence manifests sharply for $N \geq 25$. At $N = 30$, **Product** exhibits the most severe degradation (14.45s), followed by **Binary** (9.65s). Conversely, **Binomial** completes in 0.029s total, while **Commander** requires only 0.248s—roughly 58 times faster than Product. Notably, Binomial achieves the fastest pure *solve* time (0.0126s), outperforming even Commander (0.2485s).

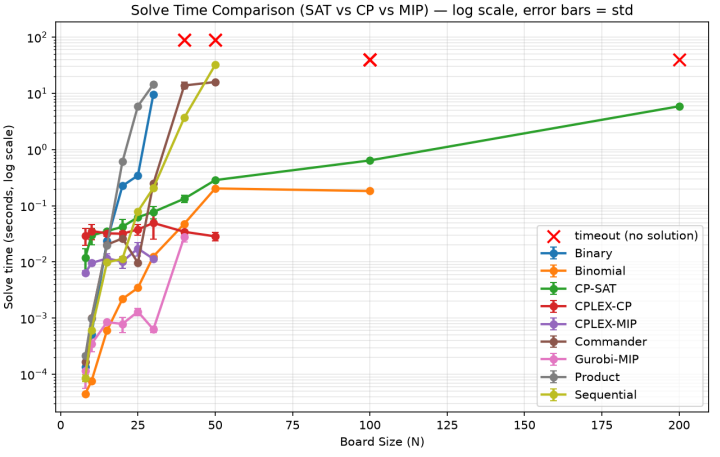


Figure 2: Solving time bottleneck analysis (logarithmic scale) in large search spaces.

Core Empirical Insight: Search Heuristics vs. Implication Chains

Bottleneck Explanation: Performance degradation is governed not merely by auxiliary variable volume, but primarily by the *implication chain length* and its alignment with unit propagation.

On the positive end, **Commander** maintains superior performance (0.248s at $N = 30$, 58× faster than Product) despite generating 1,606 auxiliary variables. This efficiency arises because commander constraints are partitioned into localized sub-groups of size $\sqrt{n}$, enabling unit propagation to immediately eliminate an entire sub-group as soon as a variable is assigned True.

On the negative end, **Product** suffers catastrophic slowdown (14.45s at $N = 30$). Arranging variables in a 2D matrix forces each queen variable to bind simultaneously to row and column auxiliaries, yielding elongated and diffuse implication chains. A decisive theoretical factor compounds this structural weakness: **the Product encoding does not preserve Generalized Arc Consistency (GAC) under unit propagation** (Table 1). Although it compresses the auxiliary-variable count to $O(\sqrt{n})$ and the clause count to $O(n)$, the loss of GAC means that unit propagation can no longer prune infeasible partial assignments early. Combined with the fact that Glucose3's VSIDS heuristic becomes disoriented, spreading its attention across the auxiliary 2D-matrix variables rather than the primary queen variables, the solver is compelled to enumerate deep into incorrect branches, producing the observed combinatorial blow-up in solving time. **Binary** closely follows (9.65s) and likewise lacks GAC. Meanwhile, **Sequential** occupies an intermediate tier (0.208s), as its sequential accumulator preserves GAC and supports unit propagation far better than the 2D grid of Product.

It is worth noting, however, that the GAC-preserving property of Sequential does not guarantee uniformly strong performance as the search space grows. Because its accumulator induces a long linear dependency chain ($s_1 \to s_2 \to \cdots \to s_{n-1}$), the learned clauses generated during conflict analysis tend to exhibit poor Literal Block Distance (LBD) scores [8]. Since Glucose3 schedules clause deletion primarily by LBD, low-quality learned clauses risk being retained or discarded suboptimally, eroding efficiency at very large instances; this mechanism is consistent with the progressive degradation of Sequential at $N = 40$ and $N = 50$.

Finally, **Binomial**—devoid of auxiliary variables—achieves the fastest solve time among SAT encodings (0.0126s). This reaffirms that CNF volume is secondary; the true computational penalty stems from heuristic dilution caused by auxiliary variables.

6.5 Fair SAT vs MIP Benchmark: First Feasible Solution

A fundamental methodological issue when comparing SAT with ILP/MIP solvers is the *termination criterion*. Pure SAT solvers address a *satisfiability* decision problem: they terminate immediately upon identifying the **first feasible solution** without regard to objective functions. Conversely, MIP solvers (e.g., Gurobi, CPLEX) inherently solve *optimization* problems, typically executing branch-and-cut to establish optimality (driving the optimality gap to zero). To verify whether this difference affects observed runtimes, we implemented a dedicated **first-solution** mode for CP/MIP solvers: Gurobi sets `SolutionLimit=1`, CPLEX MIP sets `mip.limits.solutions=1`, and CP-SAT sets `stop_after_first_solution=True`. Table 4 compares both modes on identical benchmark runs (averaged across 5 repetitions).

Surprising finding: the first-solution mode causes virtually no difference ($\Delta \approx 1.0$ across all tested instances). Two key methodological factors explain this phenomenon. First, N-Queens is fundamentally a *feasibility* problem rather than an optimization problem: the MIP formulations constructed here **contain no objective function**. Consequently, MIP solvers terminate immediately upon finding the first feasible integer solution—imposing `SolutionLimit=1` yields no acceleration. (Independent verification: Gurobi reports `SolCount=1` and `ObjBound=-inf`, confirming that no optimality proof is performed.) Second, in CP-SAT, `stop_after_first_solution` merely omits optimality bounds, but at these scales feasibility is reached almost instantaneously. *Thus, any conjecture that MIP is disadvantaged due to optimality proofs is refuted for this problem.* This further solidifies our central thesis: the performance disparity between SAT and CP/MIP does **not** arise from stopping criteria, but from fundamental differences in *modeling paradigm*—MIP branches directly on primary variables without auxiliary noise. Even under identical first-solution termination criteria, Gurobi outperforms the best SAT encoding (Binomial) by roughly 20× at $N = 30$.

Finally, non-SAT solvers demonstrate overwhelming dominance, with **Gurobi-MIP emerging as the overall fastest solver** (0.00063s solve time at $N = 30$, approximately 20× faster than Binomial and

Table 4
Comparison between default (optimization) mode and first-feasible-solution mode across board sizes N, in seconds (average of 5 runs). "Δ" represents the ratio $\text{Solve}_{\text{optimal}}/\text{Solve}_{\text{first-sol}}$; $\Delta \approx 1$ indicates that the stopping criterion causes no performance discrepancy.

Solver	N	Solve – optimal (s)	Solve – first-sol (s)	Δ
Gurobi-MIP	20	0.000 237	0.000 268	0.88
CPLEX-MIP	20	0.003 565	0.003 683	0.97
CP-SAT	20	0.015 454	0.016 063	0.96
Gurobi-MIP	30	0.000 343	0.000 335	1.02
CPLEX-MIP	30	0.005 809	0.005 784	1.00
CP-SAT	30	0.036 274	0.036 926	0.98
Gurobi-MIP	40	0.014 091	0.013 585	1.04
CP-SAT	40	0.068 825	0.069 390	0.99

Table 5
Cross-solver SAT comparison at $N = 30$ (single run), in seconds. Gen denotes the shared clause generation time.

Solver	Encoding	Clauses	Gen (s)	Solve (s)
CryptoMiniSAT5	Binomial	43 255	0.005 498	0.001 052
Glucose3	Binomial	43 255	0.033 204	0.012 604
Glucose4	Binomial	43 255	0.005 174	0.012 626
MapleSAT	Binomial	43 255	0.004 159	0.023 773
CaDiCaL	Binomial	43 255	0.009 116	0.046 083
CryptoMiniSAT5	Sequential	10 137	0.001 347	0.047 134
CaDiCaL	Sequential	10 137	0.001 168	0.129 670
Glucose3	Sequential	10 137	0.001 867	0.207 402
MapleSAT	Sequential	10 137	0.001 230	0.543 011
Glucose4	Sequential	10 137	0.001 166	0.893 541

$23,000\times$ faster than Product). **CPLEX-MIP** follows closely with 0.0113s. In the CP domain, **CPLEX-CP** (0.0494s) and **CP-SAT** (0.0776s) also comfortably outperform auxiliary-variable SAT encodings, though falling slightly behind Binomial (0.0126s). It is worth observing that Python modeling overhead for commercial solvers is nontrivial (CPLEX-MIP: 0.017s, Gurobi: 0.0075s), again reinforcing the necessity of separating formulation latency from solve latency.

6.6 Cross-Solver SAT Benchmark

To disentangle solver architecture from encoding formulation, we evaluated identical CNF models across five distinct PySAT backends: Glucose3, Glucose4, CaDiCaL, MapleSAT, and CryptoMiniSAT5. Table 5 summarizes the results at $N = 30$ for Binomial and Sequential encodings. All solvers produced valid solutions; clause generation time is invariant across solvers due to shared encoders.

The empirical data reveals a striking trend: **solver rankings reverse depending on the encoding structure**. Under Binomial encoding, **CryptoMiniSAT5** dominates (0.00105s) thanks to its advanced XOR/equivalence preprocessing and propagation engines, followed by Glucose3 (0.01260s) and Glucose4 (0.01263s), while MapleSAT (0.02377s) and CaDiCaL (0.04608s) lag behind. However, under Sequential encoding, **CryptoMiniSAT5 maintains first place** (0.04713s), followed by CaDiCaL (0.12967s), Glucose3 (0.20740s), MapleSAT (0.54301s), and Glucose4 ranks last (0.89354s). This substantiates our core thesis: *the efficiency of a (solver, encoding) pair cannot be deduced solely from encoding topology; rather, it hinges upon the synergy between clause learning heuristics and unit propagation dynamics*. Switching backends from Glucose3 to CryptoMiniSAT5 yields a $4.4\times$ speedup on Sequential (0.20740s $\rightarrow$ 0.04713s) and a

Table 6
Scalability benchmark at $N = 40$ (average of 3 runs for converging methods, 1 run for timed-out methods; 90s time limit), in seconds. "–" denotes timeout or model/license limit exceeded.

Encoding / Solver	Clauses	Variables	Gen (s)	Solve (s)
Binomial	103 540	1600	0.056 37	0.047 78
Sequential	18 312	7762	0.004 99	3.683 46
Binary	35 604	2776	0.007 87	– (timeout)
Commander	26 116	2724	0.006 25	13.822 34
Product	18 484	3998	0.004 08	– (timeout)
CP-SAT	–	–	0.001 13	0.133 59
CPLEX-CP	–	–	0.000 85	0.034 39
CPLEX-MIP	–	–	–	– (exceeded 1000-var limit)
Gurobi-MIP	–	–	0.016 53	0.028 07

Table 7
Scalability benchmark at $N = 50$ (average of 3 runs for converging methods, 1 run for timed-out methods; 90s time limit), in seconds. "–" denotes timeout or model/license limit exceeded.

Encoding / Solver	Clauses	Variables	Gen (s)	Solve (s)
Binomial	203 425	2500	0.083 85	0.203 34
Sequential	28 887	12 202	0.005 12	32.205 65
Binary	57 219	4036	0.041 51	– (timeout)
Commander	44 905	4114	0.010 87	15.933 17
Product	29 353	5910	0.005 48	– (timeout)
CP-SAT	–	–	0.002 33	0.286 26
CPLEX-CP	–	–	0.000 84	0.028 52
CPLEX-MIP	–	–	–	– (exceeded 1000-var limit)
Gurobi-MIP	–	–	–	– (exceeded license limit)

12× speedup on Binomial (0.01260s → 0.00105s)—quantitatively proving that backend architecture exerts a profound influence on identical CNF formulations. Nevertheless, a gap remains between the best SAT solver (0.04713s, CryptoMiniSAT5) and high-level solvers (CPLEX-CP: 0.04942s, Gurobi-MIP: 0.00063s at $N = 30$), reiterating that Boolean reduction incurs intrinsic overhead.

6.7 Scalability Verification at $N = 40$ and $N = 50$

To verify that these dynamics extend beyond small-to-moderate scales, we evaluated $N = 40$ under a 90-second timeout. Results (Table 6) reveal a clear inversion from $N = 30$: **Binomial emerges as the fastest SAT encoding** (0.048s solve time, 3-run average)—surpassing Sequential and vastly outperforming Commander. This underscores that as problem dimensions scale, generating $103,540$ clauses becomes negligible compared to the heuristic benefits of an auxiliary-free variable space. In contrast, both **Binary and Product exceed the 90s feasibility threshold**.

Remarkably, **Gurobi-MIP** preserves its commanding lead with only 0.028s solve time, followed by **CPLEX-CP** (0.034s) and **CP-SAT** (0.134s). An important tooling observation: **CPLEX-MIP Community Edition** reached its 1,000-variable ceiling at $N = 40$ ($1,600$ variables) and could not execute. These findings provide empirical confirmation that SAT feasibility is dictated primarily by encoding topology (auxiliary variables and implication chains) rather than problem scale alone.

At the harsher scale of $N = 50$, this trajectory is fully reaffirmed (Table 7). **Binomial** remains the fastest SAT encoding (0.203s solve time for $203,425$ clauses), whereas **Binary** and **Product** fail completely within the 90-second limit. This stark contrast offers clear proof: compressing clause counts via auxiliary variables, unless engineered with localized groups like Commander, actively destabilizes the CDCL search process.

7 Conclusion

This study provides a rigorous quantitative and theoretical investigation of exact solving methodologies for the N-Queens problem. Contrary to common intuition, compressing CNF clause counts by introducing auxiliary variables frequently induces severe performance penalties in large search spaces by corrupting CDCL branching heuristics. Crucially, the severity of degradation depends on the structural topology of the encoding rather than the raw auxiliary variable count: Commander preserves commendable performance due to localized grouping that fosters rapid unit propagation, whereas Product degrades catastrophically due to elongated, cross-matrix implication chains and, fundamentally, to its inability to preserve Generalized Arc Consistency under unit propagation—a theoretical deficit that prevents early pruning of infeasible branches (Table 1). Furthermore, this study demonstrates that for tightly constrained combinatorial problems, Constraint Programming (CP) and Integer Linear Programming (ILP) deliver vastly superior modeling abstractions and solving speeds compared to Boolean reduction. Scalability evaluations at $N = 40$ and $N = 50$ solidify these conclusions: while Binomial readily converges at $N = 50$ (despite instantiating $203,425$ clauses), inefficient auxiliary encodings (Binary and Product) break down, while CP and ILP engines resolve instances in milliseconds. Stress-testing at $N = 100$ and $N = 200$ precisely establishes the *breakdown boundary*: at $N = 100$, Binomial solves in 0.183s across $1,646,850$ clauses, whereas *every* auxiliary-variable encoding exceeds the 40s limit; at $N = 200$, Binomial itself collapses under the weight of generating $13,253,700$ clauses in 4.6s. Additionally, first-feasible-solution benchmarking reveals that termination criteria do *not* explain the SAT vs. MIP/CP gap ($\Delta \approx 1.0$ across all instances), proving that the dominance of CP and MIP is driven by direct variable branching rather than solver stopping mechanics.

Data Availability

Reproducibility Statement

All source code, environment configurations, and raw experimental datasets generated in this study are publicly hosted and openly accessible on GitHub: https://github.com/Messionly1/NQueens-SAT-Solving. The repository includes all five SAT encodings, benchmark wrappers for OR-Tools CP-SAT, CPLEX CP, CPLEX MIP, and Gurobi, alongside full CSV results, visualization scripts, and unit tests. Detailed instructions for reproducing the entire experimental pipeline are provided in the repository's README file.

References

[1] Chung, F.R.K., 1988. Labelings of graphs, in: Selected Topics in Graph Theory. Academic Press, pp. 151–168.

[2] Warners, J.P., 1998. A linear-time transformation of linear inequalities into conjunctive normal form. Information Processing Letters 68, 63–69.

[3] Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming (CP 2005). Springer, pp. 827–831.

[4] Gent, I.P., Nightingale, P., 2004. A generic polynomial time approach to the encoding of CSP into SAT. Constraints 9, 85–102.

[5] Klieber, W., Kwon, G., 2007. Efficient CNF encoding for selecting 1 from N objects, in: Fourth Workshop on Constraints in Formal Verification (CFV).

[6] Chen, J., 2010. A new SAT encoding of the At-Most-One constraint, in: Tenth International Workshop on Constraint Modelling and Reformulation (ModRef).

[7] Marques-Silva, J., Lynce, I., Malik, S., 2009. Conflict-driven clause learning SAT solvers, in: Handbook of Satisfiability. IOS Press, pp. 131–153.

[8] Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern SAT solvers, in: International Joint Conference on Artificial Intelligence (IJCAI). pp. 399–404.

[9] Rossi, F., van Beek, P., Walsh, T. (Eds.), 2006. Handbook of Constraint Programming. Elsevier.

[10] Régin, J.C., 1994. A filtering algorithm for constraints of difference in CSPs, in: AAAI Conference on Artificial Intelligence. pp. 362–367.

[11] Bertsimas, D., Tsitsiklis, J.N., 1997. Introduction to Linear Optimization. Athena Scientific, Belmont, MA.